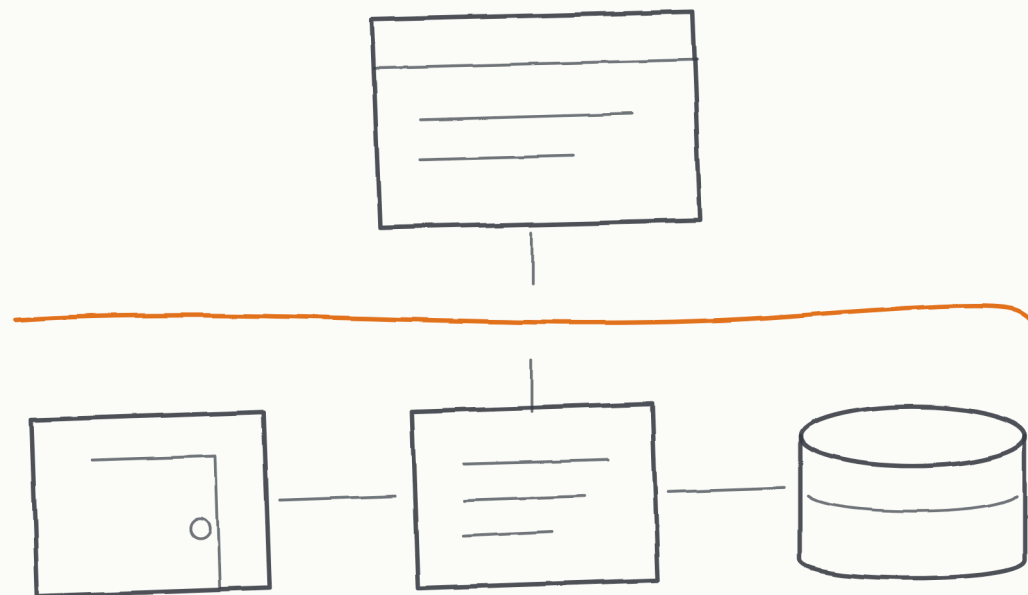


# How backend systems work

*A journey into the machinery behind every app we use.*

Instructor

*Ahmed Shamim Hassan*



# Ten classes. We are here.

| NO | CLASS                                          |
|----|------------------------------------------------|
| 01 | <i>How backend systems work</i>                |
| 02 | <i>Networking fundamentals</i>                 |
| 03 | <i>APIs and application architecture</i>       |
| 04 | <i>Authentication, authorization, security</i> |
| 05 | <i>Databases and data storage</i>              |

| NO | CLASS                                   |
|----|-----------------------------------------|
| 06 | <i>Caching</i>                          |
| 07 | <i>Real-time updates</i>                |
| 08 | <i>Queues, workers, scheduled tasks</i> |
| 09 | <i>Logging, monitoring, performance</i> |
| 10 | <i>AI-assisted development</i>          |

## *Four things, then we begin*

- ☐ *Mics on mute unless speaking.*
- ☐ *Phone on silent, face down.*
- ☐ *Close the social tabs. Yes, all of them.*
- ☐ *Question? Hand up, or drop it in the chat.*

*Ask early, ask often — every class. Questions turn a lecture into a discussion, and that is where the learning happens.*

# What we'll learn in this course

## ***The mental model***

*How the pieces fit.*

## ***The trade-offs***

*What each choice costs.*

## ***Judgement***

*Which one to pick, and why.*

We are not going to focus on any specific language, frameworks or tools.

# 01 *Shapes of software*

*Four things people call "an app". They look nothing alike.  
Underneath, they are nearly the same.*

## Four shapes we build



### Web

*Runs in a browser. Nothing to install, updates the moment we deploy.*

GMAIL · FIGMA



### Mobile

*Installed on a phone. Ships through a store, so updates are slow and optional.*

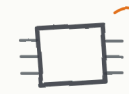
WHATSAPP · UBER



### Desktop

*Installed on a computer. Gets real access to files and hardware.*

VS CODE · PHOTOSHOP



### Device / IoT

*A thing with a chip in it. Tiny screen or none at all, often on a flaky network.*

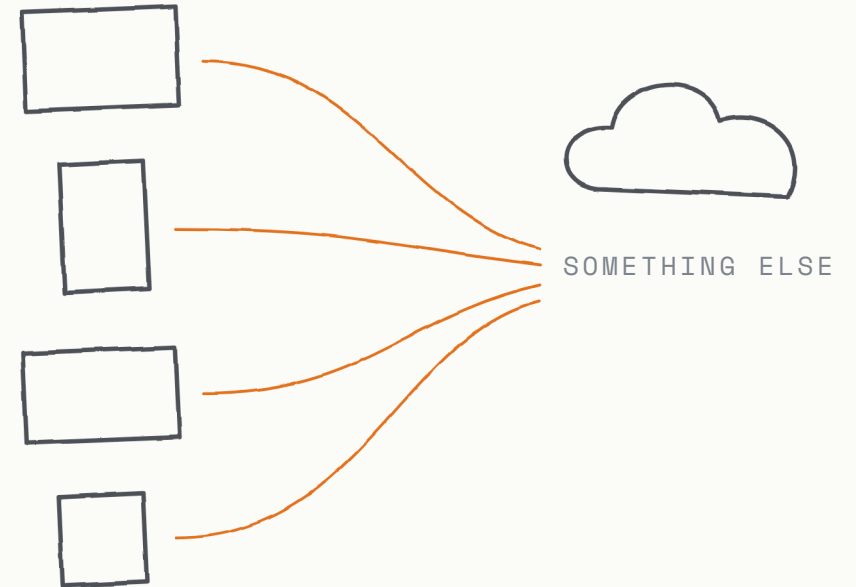
SMART BULB · CAR

*Different shapes, different rules. More on that later.*

# Underneath, all four do the same three things

- 1 Do things on the device.
- 2 Save things on the device.
- 3 Do and save things somewhere that is not the device.

Step 3 is the only one that needs a second machine. That machine is the next section.



# 02 *Client and server*

*The mental model everything else in this course is built on.*



*When we say server, do we mean this?*



*One asks. One answers.*



CLIENT

*Starts the conversation. Wants something it does not have.*

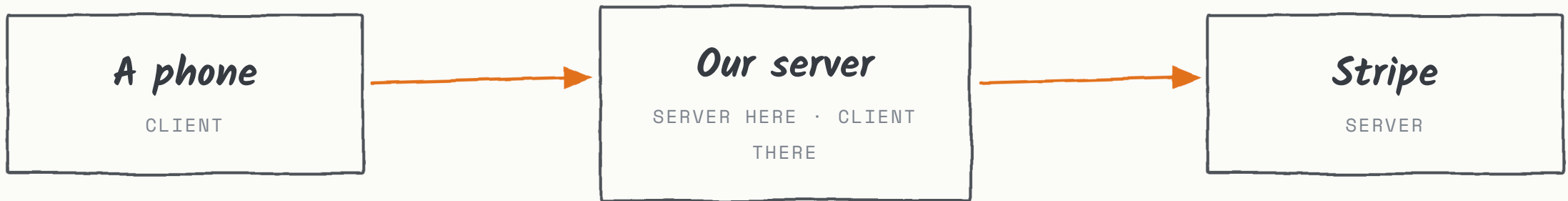
SERVER

*Waits, listens, and hands back an answer.*

*So — can a server also be a client?*



**Yes. Whoever asks is the client.**



*These are roles, not machines. The same device is a server in one conversation and can be a client in the next. So client is not a place — it is whoever is asking, right now.*

## *A different question: who can see it?*

*Client and server describe a single conversation, while frontend and backend describe which parts of the product a user is ever shown.*

### THE PART USERS SEE

*Everything the user can open and look at and tap.*

*We call it the **frontend**.*

### THE PART THEY NEVER SEE

*A machine somewhere else that does the work and keeps the data.*

*We call it the **backend**.*

## *The two splits do not line up*

- + *The frontend is always a client because it only ever asks.*
- + *The backend is the server whenever the frontend asks it for something.*
- + *That same backend becomes a client whenever it asks Stripe for something.*

*Client and server change from one conversation to the next, but a backend stays the backend even while it is being somebody else's client.*

# So why do we need a backend?

## **One shared truth**

*Everybody sees the same answer because the data lives in one place instead of on every device.*

## **A place for secrets**

*Anything the user must never hold can sit safely on a machine they can not directly look inside.*

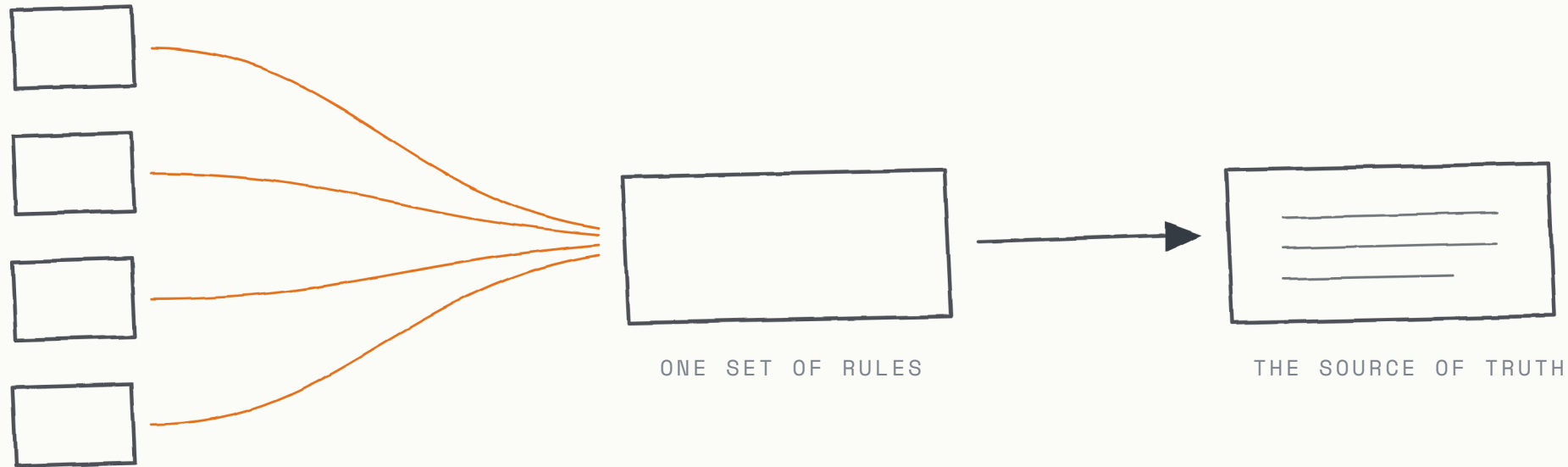
## **Work a device can not do**

*Heavy work runs on a rented machine because a phone would run out of battery long before it finished.*

## **Change without shipping**

*A rule can change on the server and reach everybody immediately, without waiting for anyone to update the app.*

*Many clients. One source of truth.*



*The backend keeps the single source of truth, so whatever one person changes is what everybody else sees next.*



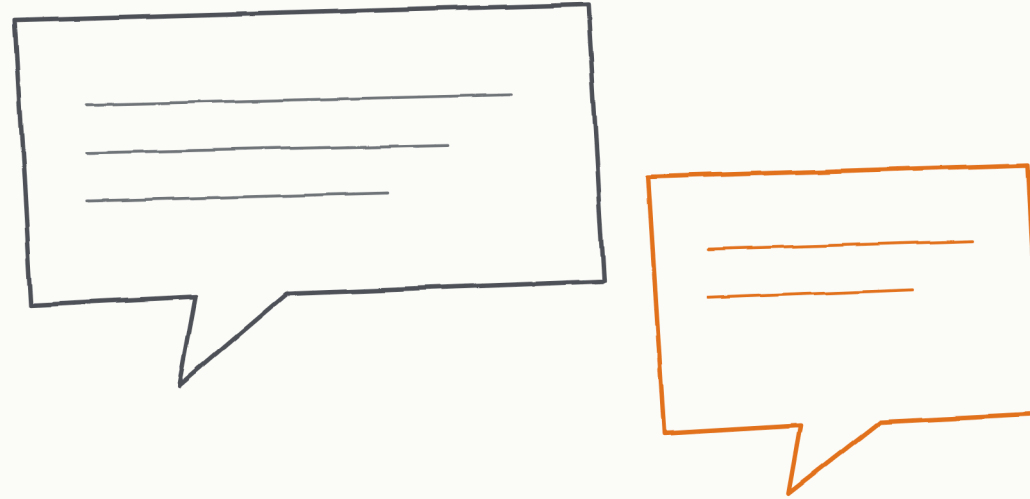
## *One note to keep for the whole course*

REMEMBER

*Anything we send to a client can be read by whoever holds that device.*

*So anything that must stay secret belongs on the backend, because users can not directly look inside it.*

*Any question about what we  
discussed so far?*



# 03 *Do we even need a backend?*

*Now that the thing has a name — sometimes the answer is no. Knowing when matters more than knowing how.*

## Every app falls into one of three buckets

### Works alone

*Never talks to anything. Turn off the internet and nothing changes.*

NO BACKEND

### Needs the network

*Useless without a connection. The interesting part lives elsewhere.*

BACKEND REQUIRED

### Works alone, syncs later

*Fine on a plane. Catches up the moment it gets a signal.*

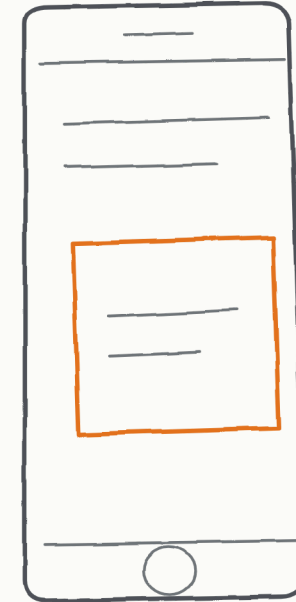
BACKEND REQUIRED

## Works alone

*Everything it needs is already on the device.  
Nobody else is in the story.*

- + *A calculator*
- + *A camera filter*
- + *A notes app that never leaves the phone*
- + *A single-player game with no scoreboard*

**Backend needed?** *No. Adding one buys nothing and costs money.*



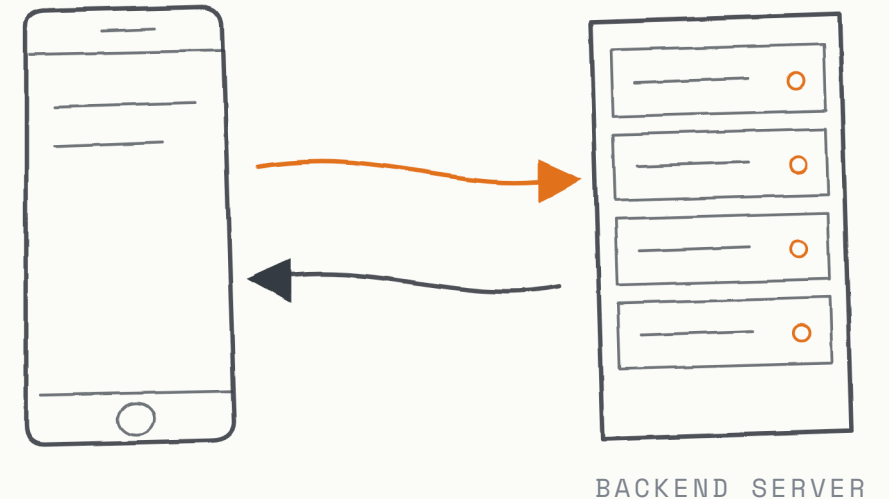
EVERYTHING IT NEEDS IS IN HERE

## Needs the network

*The screen is only a window, because the thing we came for lives on a machine the client does not own.*

- + *Chat — the message is for someone else*
- + *A feed — the posts are not ours*
- + *Banking — the balance must be the real one*
- + *Ride hailing — two strangers, one moving map*

**Backend needed?** Yes. Cut the connection and there is nothing left to look at.

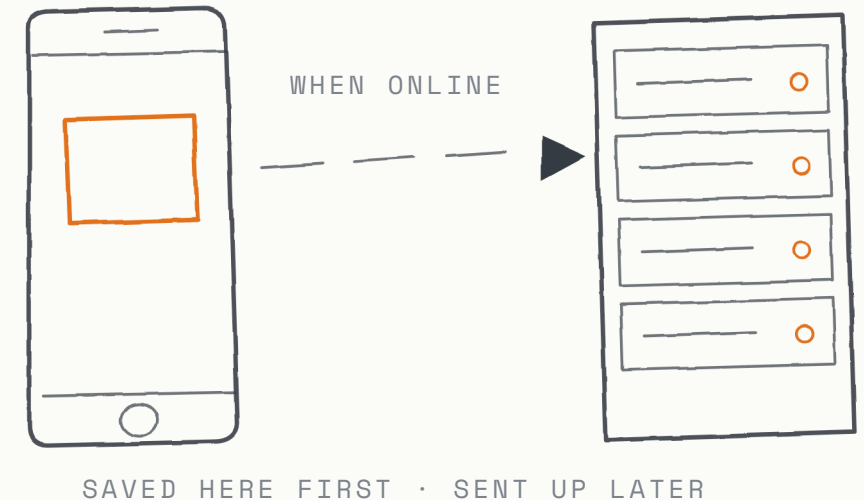


## Works alone, syncs later

*Writes to the device first, so nothing ever blocks. Sends the changes up when the network comes back.*

- + *Notes and to-dos across devices*
- + *A document edited on a plane*
- + *A fitness tracker with no signal on a run*
- + *A delivery app in a basement*

**Backend needed?** Yes — and the hardest kind. Two copies of the truth that have to agree later.



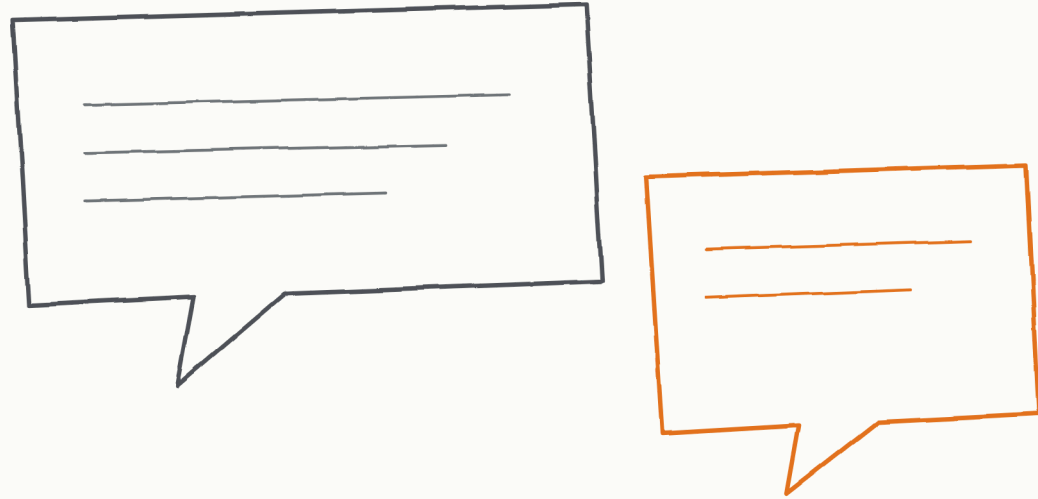
## How to decide

- ☐ Do two different people need to see the same thing?
- ☐ Must the data survive a lost or wiped device?
- ☐ Is there a secret the user must never hold — a key, a price, a rule?
- ☐ Does something need to happen while nobody has the app open?
- ☐ Do we need to change behaviour without shipping an update?

**One yes** → we need a backend. Five nos → we do not.



*Any question about what we  
discussed so far?*



# 04 *Breaking the backend apart*

*Not a language. Not a framework. A small set of parts —  
most of which we do not need on day one.*

# The whole shelf of components

## Web server

*Answers the door and hands the request inward.*

## Application

*Our rules. The part we write.*

## Database

*The memory. Survives restarts and crashes.*

## File storage

*Big things — photos, video, PDFs.*

## Cache

*A fast copy of an answer already worked out.*

## Queue + workers

*Jobs too slow to make someone wait for.*

## Scheduler

*Jobs nobody asked for, run at 3am anyway.*

## External services

*Email, payments, maps. Other people's backends.*

*This shelf is never complete, because a real backend can have many more components than these, and each one shows up only when a problem asks for it.*

## One more note to keep

REMEMBER

***Simple systems have less downtime.***

*So we add a component only when a real problem asks for it, and never to show off our engineering excellence.*

# The smallest backend that is still a backend



## WEB SERVER

*Takes connections from the internet, passes them on, hands back what comes out.*

## APPLICATION

*Our code, deciding who is asking and whether they are allowed and what the answer is.*

## DATABASE

*The only one that remembers anything, because everything else can be thrown away and rebuilt.*

# Each one solves one problem

## THE PROBLEM

---

*The same slow answer, again and again*

---

*Users waiting 30 seconds*

---

*People uploading video*

---

*Something must run at 3am*

---

## THE COMPONENT

---

*Cache*

---

*Queue + workers*

---

*File storage*

---

*Scheduler*

---

## WHAT IT COSTS

---

*Two copies, and one can be stale*

---

*One more moving piece to watch*

---

*Files and records can drift apart*

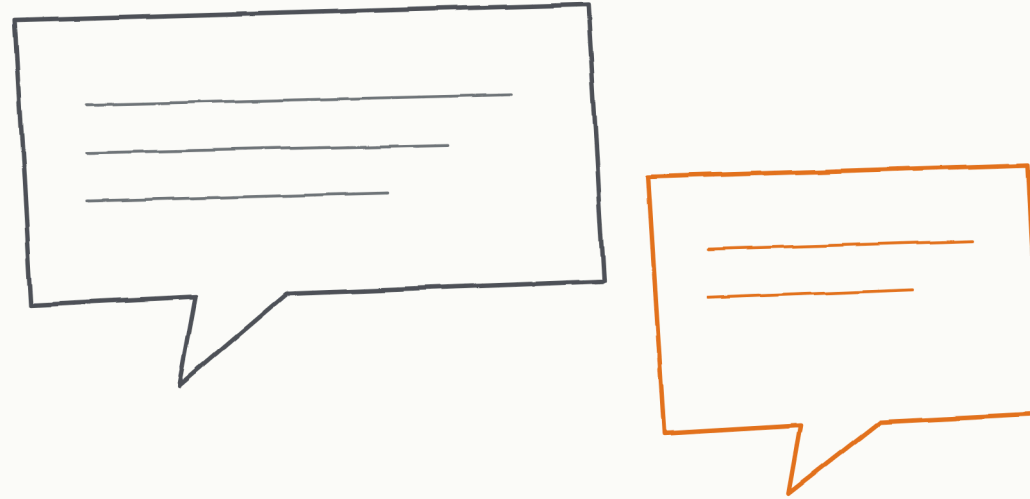
---

*It fails quietly unless somebody watches*

---

*If we cannot name the problem, we do not need the component yet.*

*Any question about what we  
discussed so far?*



# 05 *The life of one request*

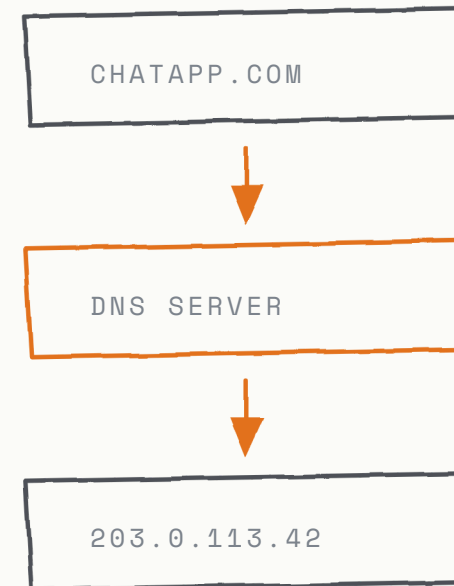
*Somebody taps a button, and eight things happen before  
the screen changes.*



## A name becomes a number

People remember *chatapp.com*, but machines only route to numbers, so a **DNS server** translates the name every single time.

**DNS is a whole system of its own**, with its own servers and caches all over the world, but today all we need is that a name goes in and an address comes out.



## Then they open a line

*Both sides say hello and agree they are ready before anything useful is sent.*



*Over HTTP the line is open and readable, while HTTPS adds one more round where both sides agree how to scramble everything, and that is the padlock.*

## The request is just text

**POST** /v1/messages HTTP/1.1

Host: api.chatapp.com

Authorization: Bearer eyJhbGciOi...

Content-Type: application/json

```
{ "to": "ayesha", "text": "on my way" }
```

**THE VERB**

What should happen to it.

**THE ADDRESS**

Which thing it happens to.

**THE PROOF**

Who is allowed to ask.

**THE PAYLOAD**

The actual stuff being sent.

Every request carries these same four parts, whatever kind of client sent it.

## Web server opens the door

*The web server holds no business rules and decides nothing about the answer, because its only job is to pass the request to the application.*



*It can hand back a logo or a stylesheet straight from disk, but anything that depends on who is asking goes inward.*

## What the application does

- 1 Who is asking? — authentication
- 2 Are they allowed? — authorization
- 3 Does the request make sense? — validation
- 4 Do the work.
- 5 Send back an answer.

**Steps 1 to 3 are only checks, and they usually take more code than the real work in step 4.**

## Database, the memory we need

*Our code asks a question in writing and waits for the rows to come back.*

```
SELECT text, sent_at  
FROM messages  
WHERE conversation_id = 4021;
```



*Our code can always be deployed again from source, but the data only lives here, which is why backups matter.*

## The response comes back

*The backend sends back a status code together with the data, so the client knows what happened and what to put on the screen.*

2xx

### ***It worked***

*Here is what was asked for, or confirmation that it is done.*

4xx

### ***The client asked wrong***

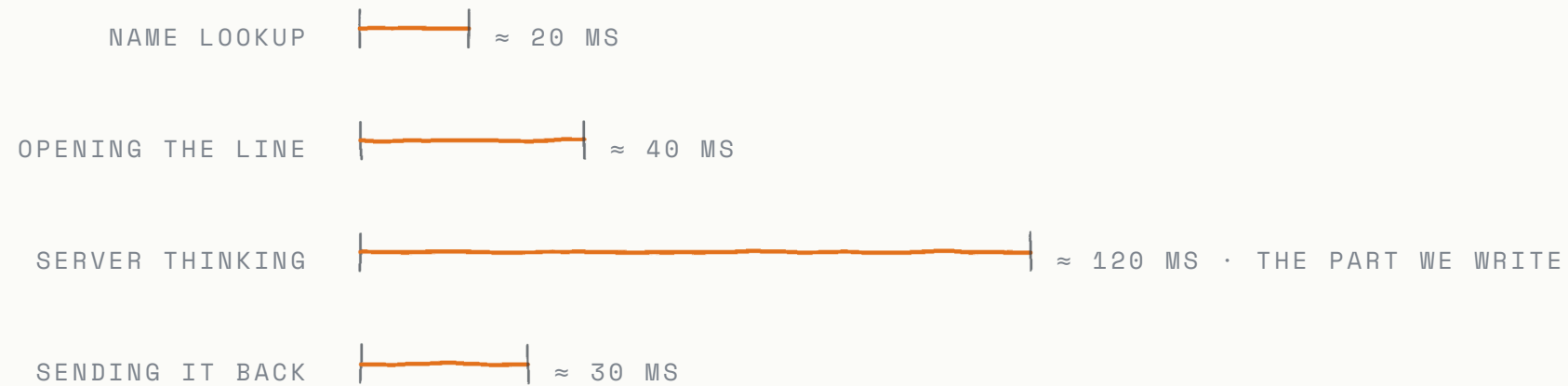
*Not logged in, not allowed, or the thing does not exist.*

5xx

### ***The server failed***

*The request was fine, but something on the server fell over.*

## Where the half second goes



*Only the third bar is ours to shrink, and most of that is spent waiting on the database.*

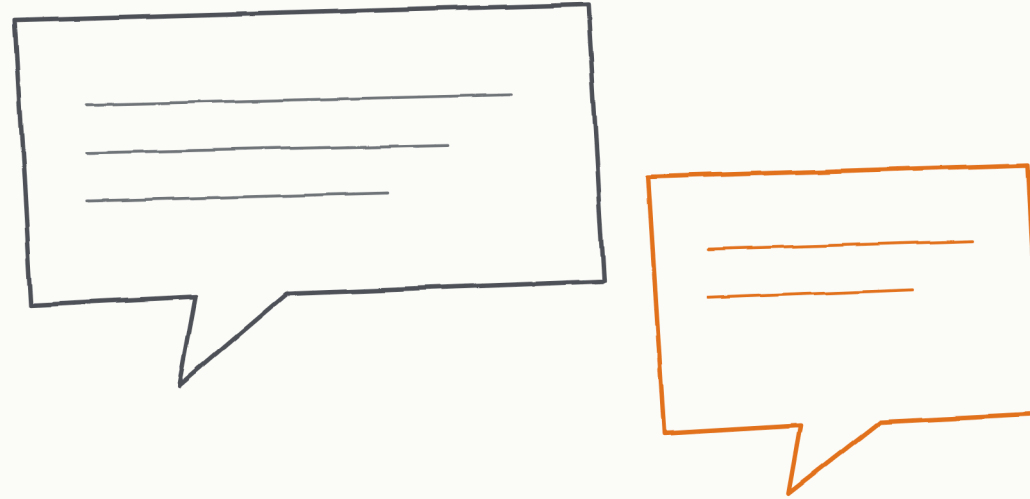


## All eight steps, end to end

- 1 *Somebody taps something with an address.*
- 2 *A DNS server turns the name into a number.*
- 3 *A line is opened to that number.*
- 4 *The request is sent down the line.*
- 5 *The web server hands it to our app.*
- 6 *Our code decides what happens.*
- 7 *The database is asked and answers.*
- 8 *A response comes back with a status code, and the screen changes.*

*Every request in every app makes this same trip, whatever shape the client is and however big the system behind it grows.*

*Any question about what we  
discussed so far?*



# 06 Servers

*A server is named after the job it does, and the web server is only one of them.*

# "Server" means two different things

## THE MACHINE

*A box in a rack in a building somewhere. No screen, no keyboard, switched on for years at a time.*

*We rent it by the month.*

## THE PROGRAM

*Software running on that machine whose whole job is to wait for connections and answer them.*

*We install it, or the host does.*

*When somebody says the server is down, ask which of the two they mean, because the answer changes who has to fix it.*

# *A server is named after its job*

## **Web server**

*Takes requests from the internet and hands back an answer.*

## **Application server**

*Runs our code and works out what the answer should be.*

## **Database server**

*Keeps the data and returns the rows we ask for.*

## **File server**

*Holds the big things like photos, video and documents.*

## **Mail server**

*Sends and receives email on our behalf.*

## **Cache server**

*Keeps a fast copy of answers that were already worked out.*

*One machine can run several of these at once, so the name describes the job it is doing rather than the box it runs on.*

# What a web server does

## **Accepts connections**

*Listens on a port and takes thousands of these at once without falling over.*

## **Serves files**

*Images, stylesheets and fonts, straight off the disk without running any code.*

## **Handles HTTPS**

*Scrambles and unscrambles the traffic, so our code never deals with it.*

## **Forwards the rest**

*Anything that needs our code to run is handed to the application.*

***It is a doorman, not a decision maker, because it never works out what the answer should be.***

# Names we will actually meet

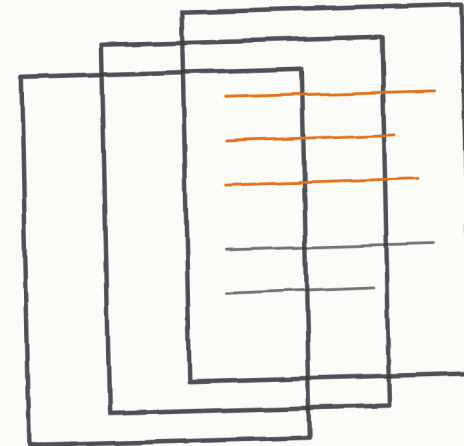
| KIND               | WHAT IT IS FOR                                                              | NAMES WE'LL SEE           |
|--------------------|-----------------------------------------------------------------------------|---------------------------|
| Static file server | Hands out files that never change. No app behind it at all.                 | NGINX · CADDY · S3        |
| Application server | Runs our code and produces answers that depend on who is asking.            | GUNICORN · PUMA · NODE    |
| Reverse proxy      | Sits in front, takes the padlock and the traffic, forwards the rest inward. | NGINX · HAProxy · Traefik |
| Managed platform   | Someone else runs the rows above and hands us a deploy button.              | VERCEL · RENDER · FLY     |

**Nginx appears twice** because one program can play more than one of these roles, which is the main reason the names are confusing.

## Sometimes files are enough

*When every visitor sees the same page, there is no code to run and nothing to look up.*

- + *A documentation site*
- + *A landing page or a portfolio*
- + *A blog written ahead of time*
- + *Anything built once and published*

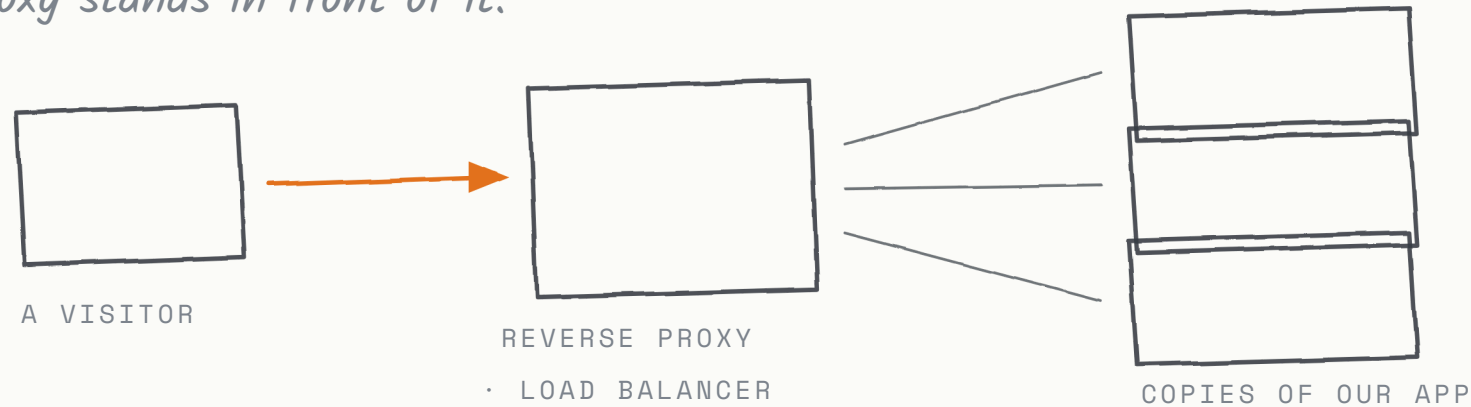


*No application and no database means almost nothing to break, and hosting costs close to nothing.*



## When our code has to run

*As soon as the answer depends on who is asking, files are not enough, so an application server runs our code and a reverse proxy stands in front of it.*



IT TAKES WORK OFF OUR CODE

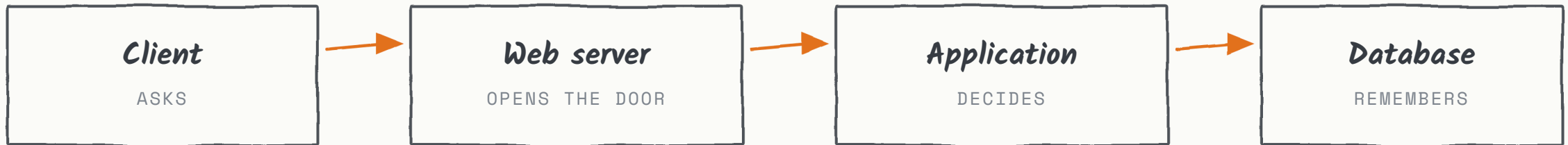
*It handles HTTPS and hands out the files on disk, so our application never has to.*

AND IT BALANCES THE LOAD

*It shares the requests across several copies of our app, so one can die and the rest carry on.*

## What we covered today

- 1 Web, mobile, desktop and IoT are all clients.
- 2 Client and server, frontend and backend.
- 3 Five questions decide if we need a backend.
- 4 Web server, application, database.
- 5 DNS, connection, HTTP request, status code.
- 6 Web, application, database, cache and mail servers.

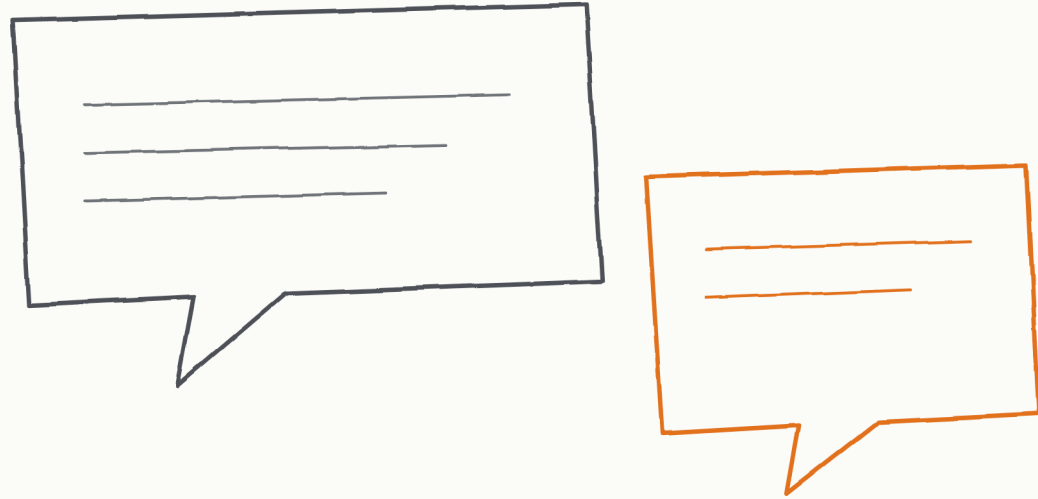


## *Can we answer these without the slides?*

- ☐ *Name an app that genuinely needs no backend, and say why.*
- ☐ *What are the three parts of the smallest real backend?*
- ☐ *Why can the browser never be trusted to check a password?*
- ☐ *Why is a backend still a client sometimes?*

*Say them out loud to somebody who does not write code. That is the real test.*

*Any questions before we finish?*



## One small thing to do

*Pick any app on your phone. On paper, draw what happens between the tap and the screen changing.*

- + *Mark which parts run on the phone and which do not.*
- + *Mark the one place the data is actually kept.*
- + *Circle anything hard to explain. Bring the circles.*

**Next up: networking.** We take the first arrow in the drawing and pull it apart — how a message actually crosses the world.

# Thank you

*See you next week, when we pull the first arrow apart.*

